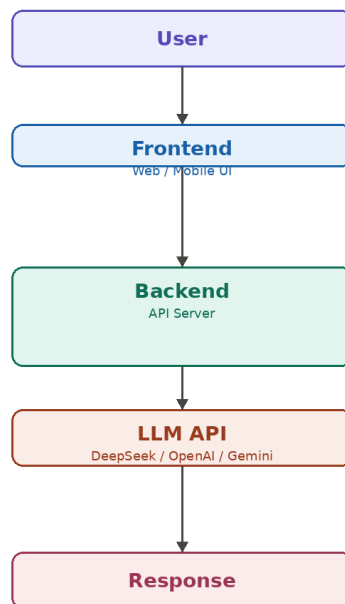


AI Application Architecture

Introduction

An AI application architecture is the structural blueprint that defines how the user interface, backend logic, and underlying AI model interact to deliver intelligent functionality. It specifies how data flows from user input to the AI model and back, while addressing security, scalability, and fault tolerance. A well-designed architecture ensures AI applications are reliable, secure, maintainable, and scalable as large language models become central to modern software products.

Architecture Diagram



Request Flow

- The user enters a query in the frontend (web or mobile UI).
- The frontend validates the input and sends it as an HTTP request to the backend.
- The backend authenticates the request and applies business logic (prompt formatting, context injection), building a payload for the LLM.
- The backend calls the LLM API over a secure HTTPS connection with that payload.

Response Flow

- The LLM API returns the generated output to the backend, typically as JSON.
- The backend parses the response, applies post-processing (filtering, logging, formatting), and forwards it to the frontend.
- The frontend renders the response for the user in a readable format.
- A well-optimized round trip completes within a few seconds to preserve a smooth user experience.

Error Handling

Robust error handling is essential for production-grade AI applications. Key failure scenarios and mitigations:

Error	Cause	Suggested solution
Invalid user input	Empty or malformed query	Validate on frontend and backend; show a clear error message
API timeout	LLM takes too long to respond	Set a timeout threshold; retry with exponential backoff
Authentication failure	Invalid or expired API key	Validate credentials before sending; auto-refresh tokens
Rate limit exceeded	Too many requests in a time window	Throttle/queue requests; notify the user
Network failure	Loss of connectivity	Retry logic; graceful fallback (cached response, offline message)
Internal server error	Unexpected backend failure	Centralized logging and monitoring; fallback error response

API Integration

- **Authentication** — API key authentication, sent via the request header.
- **Request format** — Requests sent as HTTP POST calls with a JSON body containing the prompt and parameters.
- **Response format** — Responses returned as JSON, containing the generated text and metadata such as token usage.
- **Parsing** — The backend extracts the relevant content field from the JSON response.
- **Display** — The frontend renders the parsed content to the user.
- **Security best practices** — Store API keys in environment variables, enforce HTTPS, and validate/sanitize all input.

Conclusion

An effective AI application architecture integrates the user interface, backend logic, and LLM API into a cohesive, secure, and scalable pipeline. By clearly defining request and response flows, implementing comprehensive error handling, and following API integration best practices, engineers can build AI-powered applications that are reliable and production-ready.